

notes

Taking notes in class...

Markdown → PDF (macOS)

This repo uses `md-to-pdf` (headless Chromium) plus `pdf-theme.css` so exports look clean on paper: system fonts, readable code blocks, and sensible margins.

Setup (once)

1. Install `Node.js` (LTS is fine), which includes `npm`.
2. In this folder:

```
npm install
```

Commands

Command	What it does
<code>npm run pdf -- path/to/note.md</code>	Writes <code>path/to/note.pdf</code> next to the Markdown file.
<code>npm run pdf -- a.md b.md</code>	Converts several files in one go.
<code>npm run pdf:watch -- path/to/note.md</code>	Rebuilds the PDF whenever that file changes (stop with Ctrl+C).

Examples:

```
npm run pdf -- README.md
npm run pdf -- ./lectures/week3.md
npm run pdf:watch -- ./lectures/week3.md
```

PDF-only text transforms

`scripts/md2pdf.mjs` runs **before** Markdown is turned into HTML. It only touches text **outside** fenced triple-backtick code blocks. Your `.md` files on disk are unchanged; the PDF sees the transformed string.

Order of steps: Greek letter names → set/membership phrases (below) → caret superscripts → cleanup inside `<sup>...</sup>` tags.

Greek letter names → Unicode

English names are found case-insensitively; the **first character of the matched word** picks uppercase vs lowercase Greek (e.g. Σ and $\Sigma \rightarrow \Sigma$, $\sigma \rightarrow \sigma$):

You write	PDF gets
$\alpha \dots \omega$	$\alpha \dots \omega$ (lowercase Greek)
$A \dots \Omega$	$A \dots \Omega$ (uppercase Greek)

Names supported: $\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \eta, \theta, \iota, \kappa, \lambda, \mu, \nu, \xi, \omicron, \pi, \rho, \sigma, \tau, \upsilon, \phi, \chi, \psi, \omega$.

The first character of the word decides upper vs lower Greek: $\Sigma \rightarrow \Sigma$, $\sigma \rightarrow \sigma$, $\Sigma \rightarrow \Sigma$. Names must appear as **whole words** (word boundaries), so ϵ in prose becomes ϵ , but a variable like `myepsilon` is left alone.

Discrete phrases → symbols

Phrases are matched as substrings (case-insensitive). Longer phrases are applied first so negations stay correct.

You write (examples)	Symbol
$\notin, \notin$	$\notin$
$\in, \in$	$\in$
$\subseteq$	$\subseteq$
$\cap$	$\cap$
$\cup$	$\cup$

Typical patterns: $\text{if } A \cup B, L \subseteq \Sigma^{<\sup>*}</sup>, x \in S$.

Caret superscripts ($\wedge \dots$)

A caret not immediately after `[` (footnote style) or `\` (escape) becomes an HTML superscript for the PDF.

Form	Example	Result (idea)
Digits	<code>x<sup>2</sup></code> , <code>Σ<sup>10</sup></code>	superscript digits
Single Latin letter	<code>x<sup>n</sup></code>	one letter, not followed by another letter
Braced text	<code>x<sup>n+1</sup></code> , <code>Σ<sup>*</sup></code>	anything in <code>{...}</code>
Word	<code>Σ<sup>u</sup></code> , <code>Σ<sup>u</sup></code>	u in superscript
Single math / operator character	<code>Σ<sup>*</sup></code> , <code>x<sup>+</sup></code> , <code>A<sup>C</sup></code>	<code>*</code> <code>+</code> <code>u</code> <code>∩</code> <code>≤</code> <code>≥</code> ... and related symbols (see script)

Inside `{...}` or word superscripts, `u` and `n` are normalized to `u` and `∩`. You can also type those Unicode symbols directly after `^` as a one-character superscript.

`[^label]` footnote markers and `\^` (escaped caret) are **not** treated as superscripts.

Implementation detail: [scripts/md2pdf.mjs](#) (`GREEK_BY_NAME` , `SET_OPERATOR_PHRASES` , `CARET_EXPONENT`).

Styling the PDF

Edit `pdf-theme.css` at the project root. The script always applies that stylesheet on top of the HTML that `md-to-pdf` generates from your Markdown.

Use normal CSS aimed at **print**: `@page` margins, `body` typography, `pre / code` , `table` , `blockquote` , etc. After changes, run `npm run pdf -- your-file.md` again.

Markdown commands for styling

Below is a concise reference for what you can write in `.md` files. The converter uses **Marked** with GitHub-flavored Markdown–style features (tables, task lists, strikethrough, etc.).

Headings

```
# Largest title
## Section
### Subsection
#### Smaller heading
```

Use more `#` symbols for deeper levels (up to six levels with `#####`).

Emphasis

You write	Result
<code>**bold**</code> or <code>__bold__</code>	bold
<code>*italic*</code> or <code>_italic_</code>	<i>italic</i>
<code>***bold italic***</code>	<i>bold italic</i>
<code>~~strikethrough~~</code>	strikethrough (GFM)

Paragraphs and line breaks

- One blank line starts a new paragraph.
- End a line with **two spaces** before the newline, or use a **backslash** at end of line, for a soft line break inside a paragraph.

Lists

Bulleted:

- Item one
- Item two
 - Nested item

Numbered:

1. First
2. Second
 1. Nested step

Links and images

```
[Link text](https://example.com)
![Description of image](./figures/diagram.png)
```

Paths to images are resolved relative to the **Markdown file's folder**, which helps for lecture assets next to your notes.

Code

Inline: ``const x = 1``

Fenced block with optional language for highlighting:

```
```\javascript
function hello(name) {
 return `Hello, ${name}`;
}
```
```

Blockquotes

```
> Important definition or quote.
> Multiple lines can continue with `>`.
```

Horizontal rule

```
---
```

(or `***` on its own line)

Tables (GFM)

| Column A | Column B |
|----------|----------|
| foo | bar |
| baz | qux |

Alignment: `|:---|` left, `|:---:|` center, `---:|` right.

Task lists (GFM)

- [] Todo item
- [x] Done item

Escaping

Prefix special characters with `\` when you want them literally, e.g. `\*not italic\*`.